



Qualidade e Testes de Software em um contexto de programação com LLM

Orlando Saraiva Jr

Mestre em Tecnologia
FT - Unicamp - 2013

MBA em Gestão Estratégica de Negócios
Unifian - 2008

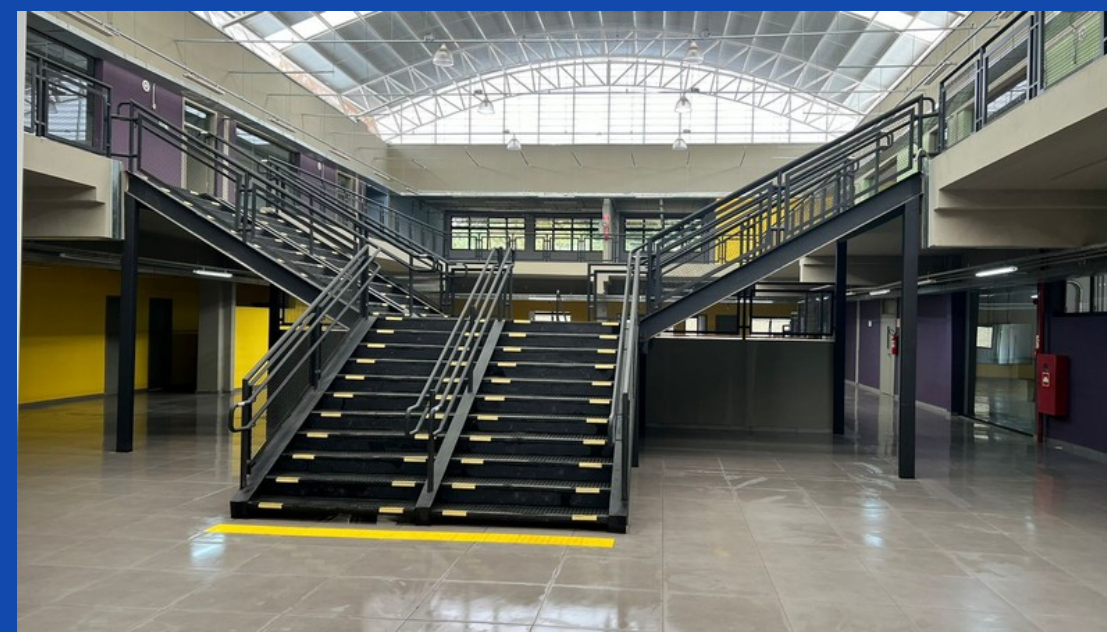
Graduação em Tecnologia em Informática
CESET - 2005





Assistente de Suporte acadêmico
Unesp Rio Claro desde 2007

Docente na FATEC Rio Claro
desde 2025



Docente na FATEC Araras
desde 2018



Qualidade e Testes de Software

O que é qualidade ?

Existem várias definições de qualidade, como a definição proposta por **Philip Crosby** como “**conformidade com os requisitos**”.

Esta definição não leva em diferença intrínseca na qualidade dos produtos em consideração ao julgar a qualidade do produtos.

Por exemplo, esta definição pode sugerir que um carro Mercedes é do tipo mesma qualidade de um carro Fusca.

Além disso, a definição não considera se os requisitos são realmente apropriados para o produto.

Juran define qualidade como “**adequação ao uso**”, e esta é uma definição melhor, embora não forneça um mecanismo para julgar a melhor qualidade quando dois produtos são igualmente aptos para serem usados.

Gerenciamento da Qualidade

Na Idade Média, um artesão era responsável pelo desenvolvimento completo de um produto desde sua concepção até a entrega ao cliente.

Isso levou a um forte sentimento de orgulho e propriedade da qualidade do produto, e os aprendizes juntaram-se aos artesãos para aprender as habilidades do ofício.



A Revolução Industrial trouxe uma mudança a este paradigma tradicional, e o trabalho tornou-se altamente organizado com trabalhadores responsáveis por uma parte específica do processo de fabricação.



O senso de propriedade e o orgulho do trabalho em produto foram diluídos, pois os trabalhadores passaram a ser responsáveis apenas por sua parcela o produto, e não a qualidade do produto como um todo.



A gestão da qualidade total (TQM) é uma abordagem moderna para a gestão da qualidade, e essa filosofia de gestão envolve foco no cliente, melhoria de processos, desenvolver uma cultura de qualidade dentro da organização e desenvolver um programa de verificação e análise.



Enfatiza que os clientes têm direitos e expectativas de qualidade, que devem ser satisfeitas, e que todos na organização é um cliente e tem clientes.



Qual minha postura quando o assunto
é qualidade de software ?

O controle de qualidade de software se preocupa com as atividades para garantir que o produto final satisfaz os requisitos funcionais e não funcionais e é adequado para o propósito.

Isto inclui inspeções e testes para verificar se as entregas produzidas satisfazem seus requisitos.

As inspeções normalmente consistem em uma revisão formal de uma entrega por especialistas independentes, e o objetivo é identificar defeitos no produto de trabalho e para fornecer confiança em sua correção.

O teste de software consiste em técnicas de teste de “caixa branca” ou “caixa preta” e as atividades de teste incluem testes de unidade, sistema, desempenho e aceitação.

O teste é bastante metódico e inclui um conjunto abrangente de casos de teste manuais ou automatizados. As atividades de verificação e validação envolvem a execução dos testes definidos e a correção de quaisquer testes falhos ou bloqueados.

O custo de correção de um defeito está relacionado à fase em que ele é detectado no ciclo de vida.

Os erros detectados nas fases iniciais são os menos dispendiosos para corrigir e os defeitos detectados posteriormente tornam-se cada vez mais caros para corrigir.

Programação com LLMs

Programar com o apoio de LLMs desloca parte da capacidade do modelo para memória, skills, protocolos e o harness que o envolve — o ambiente de execução que sustenta o agente. Cada uma dessas peças passa a ser, na prática, um artefato cognitivo, que reorganiza a tarefa para o modelo resolvê-la de forma mais confiável.

Isso desloca também a responsabilidade da **qualidade e dos testes**: eles deixam de ser uma etapa final e passam a fazer parte da própria engenharia do agente.

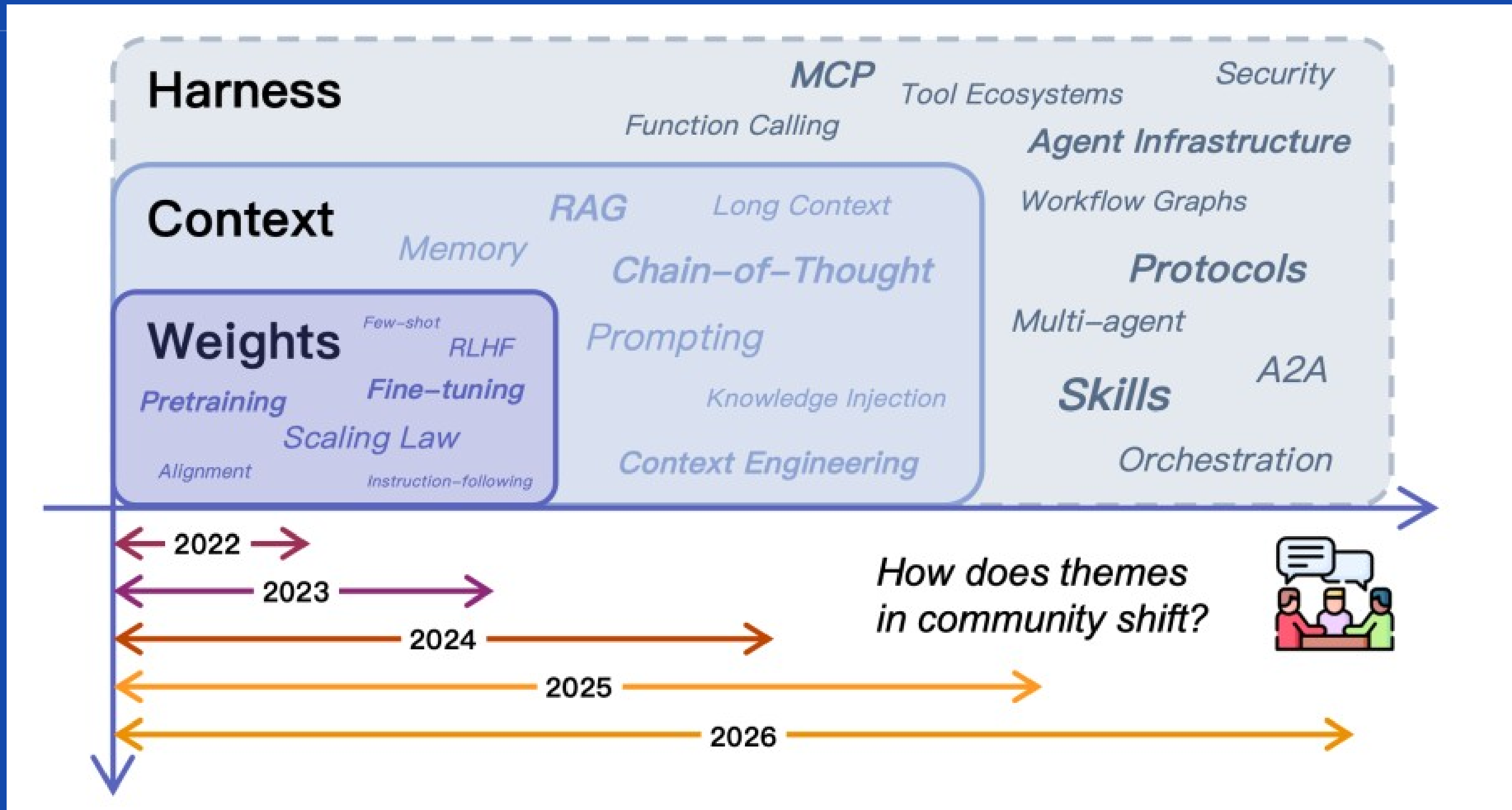
Entre 2020 e 2022, a capacidade dos LLMs era tratada quase inteiramente como propriedade dos pesos do modelo. O pré-treinamento em grandes corpora comprimiu conhecimento e padrões de raciocínio diretamente nos parâmetros, como em GPT-4, Gemini e DeepSeek-V3.

O problema é que conhecimento acoplado a um artefato estático é **difícil de atualizar, compor e auditar**: não existem módulos inspecionáveis para isolar e testar isoladamente.

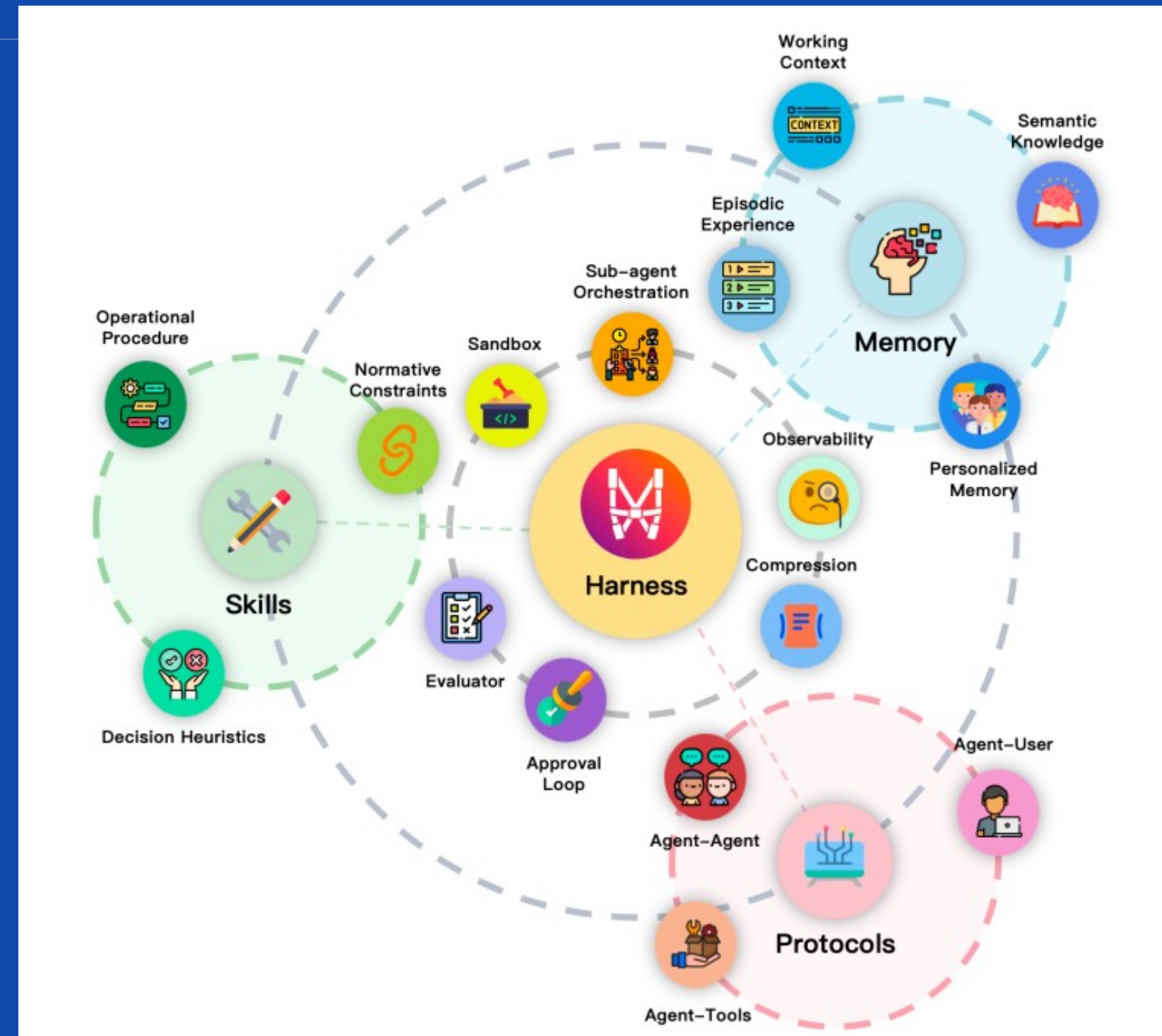
Entre 2022 e 2023, a atenção migrou dos pesos para o desenho do contexto. Técnicas como few-shot prompting, chain-of-thought, ReAct e RAG mostraram que o comportamento do modelo podia mudar substancialmente sem qualquer atualização de parâmetros.

O limite dessa fase é conhecido: janelas de contexto são finitas e o fenômeno **“lost in the middle”** reduz a precisão quando a informação relevante fica no meio de prompts longos.

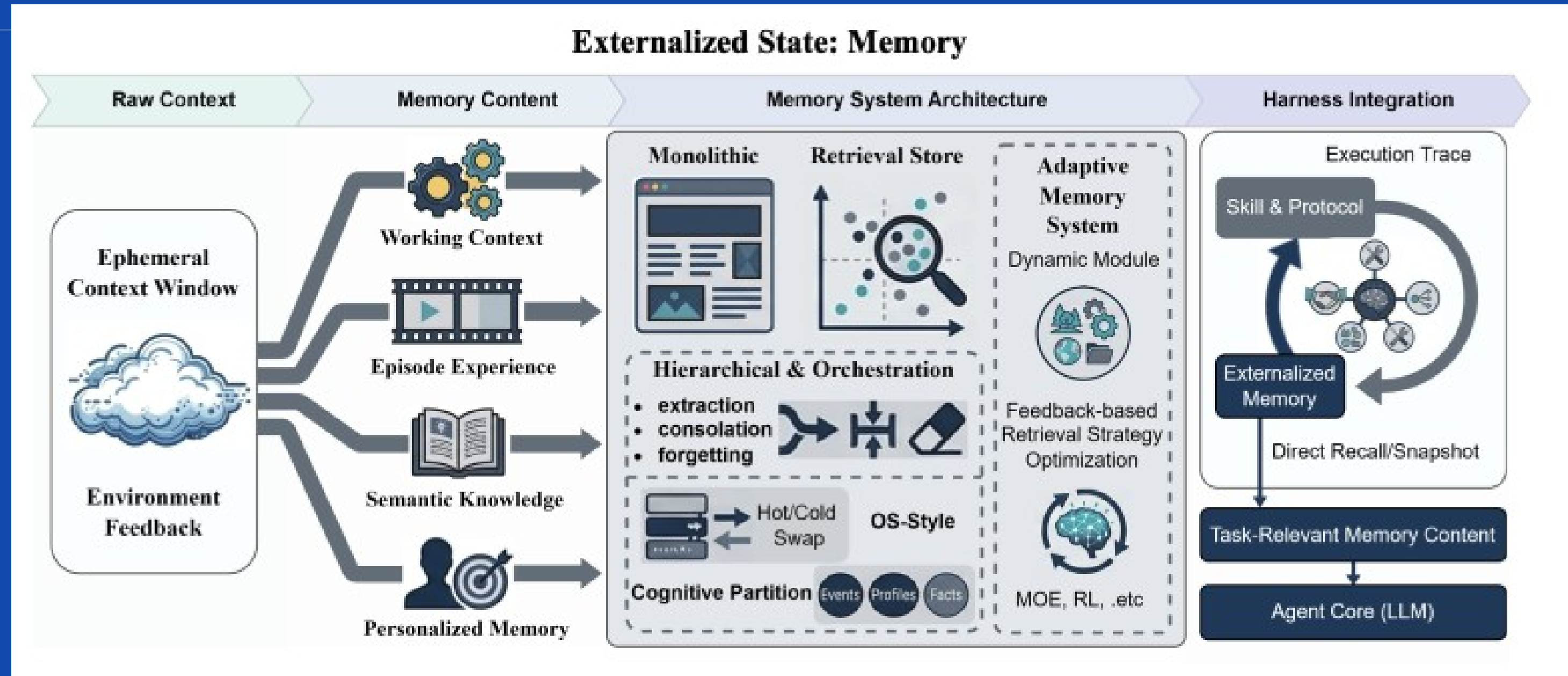
A partir de 2023, o centro de gravidade migra para o harness: memória, skills, protocolos e orquestração multiagente passam a decidir a confiabilidade dos agentes.



O harness organiza três eixos de externalização — memória, skills e protocolos — orbitando um núcleo de orquestração, sandboxing, observabilidade e avaliação.



A memória externaliza o estado do agente ao longo do tempo: do contexto de trabalho efêmero à experiência episódica e ao conhecimento semântico persistente.



Em agentes de codificação, o harness passa a incluir arquivos, shell, controle de versão e testes automatizados. Skills documentam as convenções do projeto; protocolos garantem chamadas corretas a test runners; o sandbox isola a execução com segurança — como em SWE-agent e OpenHands.

Nesses ambientes, requisitos de teste e rastreabilidade deixam de ser checagem posterior e passam a ser **precondição da própria tarefa.**

Um agente que age sem deixar rastro inspecionável não pode ser depurado, auditado ou melhorado. A observabilidade registra cada chamada de modelo, uso de ferramenta e decisão, fechando o loop entre execução e aprendizado.

Pontos de aprovação — **approval gates** — definem onde a supervisão humana intervém: antes da ação, depois da execução ou apenas quando um gatilho de risco é detectado.

Externalizar também cria novos riscos: sobrecarga cognitiva por excesso de memória e skills, envenenamento de memória, injeção de skills maliciosas e falsificação de protocolo. Por isso, gates de revisão, rastreamento de proveniência e testes de regressão tornam-se parte da infraestrutura, não apenas do checklist final.

A qualidade de software com LLMs se move, assim, da avaliação pontual para a **engenharia contínua do harness** — testes, sandboxes e observabilidade como parte estrutural do agente.

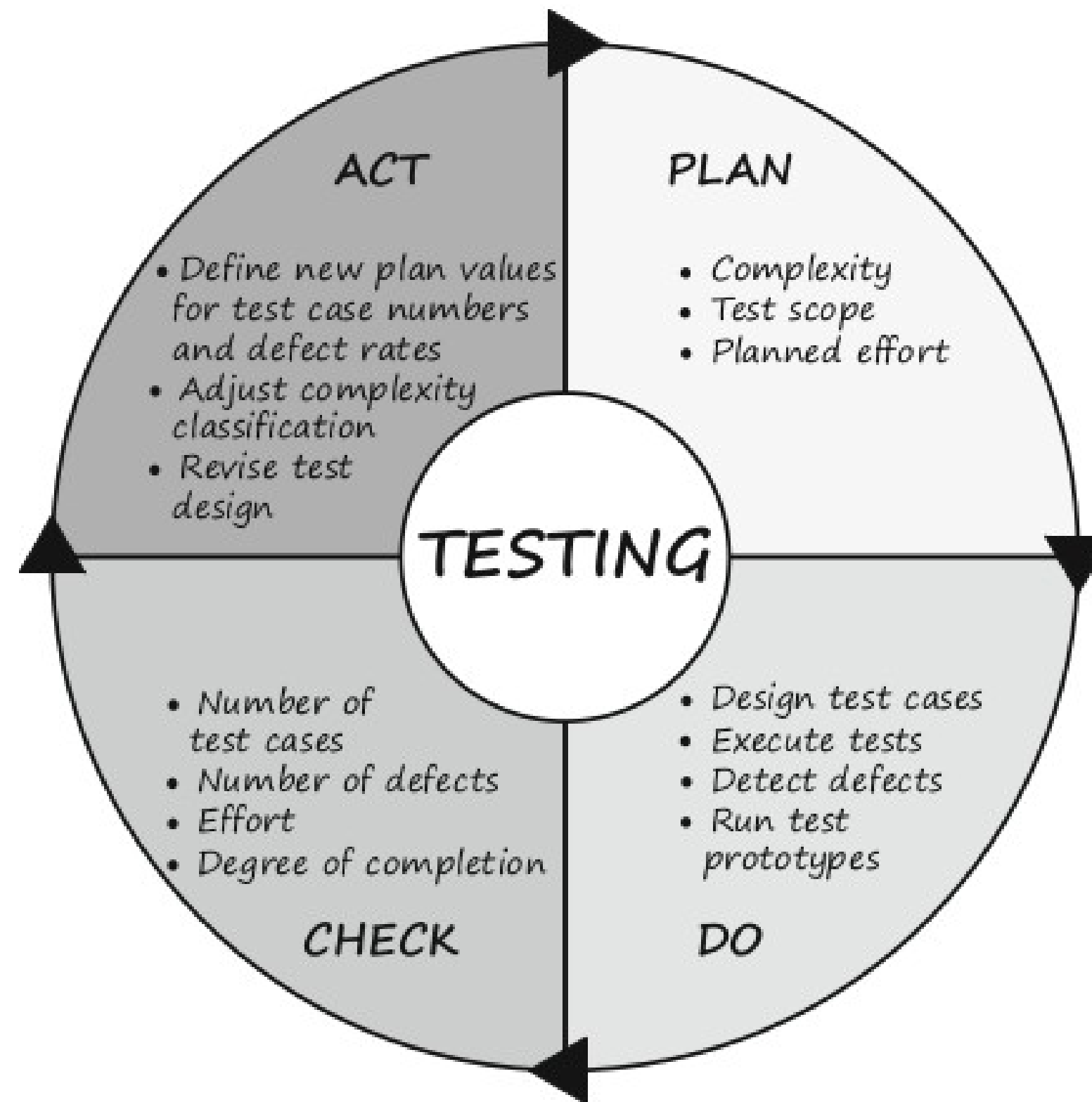
Ciclo PDCA

Ciclo PDCA

O ciclo PDCA — Planejar, Fazer, Verificar, Agir — é o loop de controle que mede resultados, avalia desvios, toma contramedidas e verifica a eficácia da correção. Também chamado de **ciclo de Deming** (Deming, 1982), costuma ser informal em projetos ágeis, sustentado pela comunicação constante dentro da equipe.

Em agentes de LLM, o mesmo loop reaparece na **observabilidade do harness**: medir, detectar o desvio, corrigir e validar a correção antes de seguir em frente.

Ciclo PDCA



Manfred Baumgartner · Martin Klöckner ·
Christian Mastnak · Helmut Pichler ·
Richard Seidl · Siegfried Tanczos

Agile Testing

The Agile Way to Quality

Hands on !

Perguntas ?



<https://orlandosaraivajr.github.io/>

orlando.saraiva@unesp.br

orlando.nascimento@cps.sp.gov.br

Bibliografia

BAUMGARTNER, Manfred et al. Agile testing: the agile way to quality. Cham: Springer, 2021.

ZHOU, Chenyu et al. Externalization in LLM agents: a unified review of memory, skills, protocols and harness engineering. [S. l.]: arXiv, 2026. Disponível em: <https://arxiv.org/abs/2604.08224>. Acesso em: 11 set. 2026.